
Data Structures

BS (CS) _Fall_2024

Lab_10 Tasks



Learning Objectives:

1. Binary Tree

Lab Tasks

Submission Instructions

1. Create a separate **.h** and **.cpp** file for each class.
2. Now create a new folder with name *ROLLNO_SEC_LAB09* e.g. **i23XXXX_A_LAB07**(Also name your **cpp** file like this.)
3. Move all of your files to this newly created directory and compress it into a **.zip file**. Return the test file as well.
4. Now you have to submit this zipped file on Google Classroom.
5. If you don't follow the above-mentioned submission instruction, you will be marked **zero**.
6. Plagiarism or AI generated code in the Lab Task will result in **zero** marks in the whole category.

Question 1

Write a C++ code to implement the node-based implementation of a binary tree. You must implement the following functions.

1. You will need to make a node class that has the following parameters.
2. Now you need to make a Binary tree class that will contain the root of the tree.
3. Void insert(int data)

You can insert the node in the following manner if you want to insert the node of the left child. Following is an example of how you will add a node to left of root node

```
//Step 1: Make a new tree node;
Node * newNode = new
Node(data);
//Step 2: Make current node that points to root
Node * currentnode = new Node(data);
//Step 3: Adjust the new node to the left of the current
nodecurrent->left = newNode
```

The tricky part in this question is that you have to add nodes in such a way that a complete binary tree is formed.

4. `Void InoderTraversal(Node * treeNode):`
This is a recursive function that will be called to traverse the tree in order.
5. `Void PreorderTraversal(Node * treeNode):`
This is a recursive function that will be called to traverse the tree in order.
6. `Void PostoderTraversal(Node * treeNode):`
This is a recursive function that will be called to traverse the tree in order.
7. `Void Display()`
This function will display elements with root of tree.

Question 2

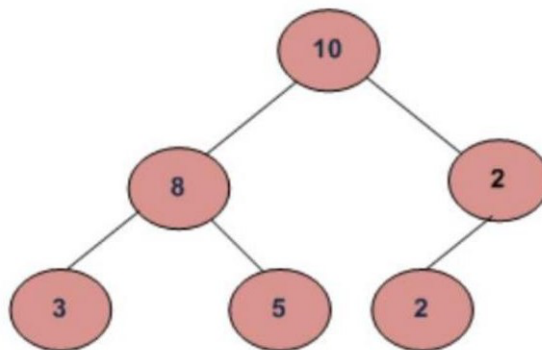
You are given a binary tree, and a target sum. Your task is to determine if there exists a path from the root to a leaf in the tree such that the sum of values along the path is equal to the target sum. Write a function `hasPathSum` to solve this problem. The function should take as input the root of the binary tree and the target sum and return `True` if such a path exists, and `False` otherwise.

Example:

Target

sum:14

Tree:



Output:

True

Explanation:

The Path 10->2->2 has a total sum of 14, so the function will return true.